



COMPREHENSIVE GUIDE

The Claude Code

Marketing Playbook

From Beginner to Advanced: Mastering AI-Powered Marketing with
Claude Code and Agent Skills

BASED ON MARTECH AI & MARKETINGSKILLS

2026 Edition

THE CLAUDE CODE MARKETING PLAYBOOK

From Beginner to Advanced: Mastering AI-Powered Marketing
with Claude Code and Agent Skills

A comprehensive guide synthesizing insights from Charlie Hills' MarTech AI newsletter
and Corey Haines' marketingskills repository.

2026 Edition

Table of Contents

Module 1: Introduction	5
1.1 Why This Playbook Exists	5
1.2 Who This Playbook Is For	5
1.3 What Readers Will Achieve	6
Module 2: Concept Foundations	6
2.1 What Is Claude Code?	6
2.2 What Are Agent Skills?	7
2.3 The MCP Protocol	7
2.4 Context Windows and Token Economics	7
Module 3: System Overview	8
3.1 The Claude Code Architecture	8
3.2 The Agent Skills Ecosystem	9
3.3 How the Pieces Connect	10
Module 4: Step-by-Step Implementation	10
4.1 Beginner Workflow	10
4.2 Intermediate Workflow	12
4.3 Advanced Workflow	13
Module 5: Real-World Use Cases	14
5.1 Lead Scraping and Enrichment	14
5.2 Proposal Generation	15
5.3 Website Generation from Screenshots	15
5.4 Video Generation with Remotion	16
5.5 SEO Audit and Optimization	16
Module 6: Tools and Stack	16
6.1 Core Platform	16
6.2 MCP Servers	17

6.3 Marketing Tool Integrations	17
6.4 Composio for OAuth-Heavy Platforms	17
Module 7: Framework Breakdowns	18
7.1 The CLAUDE.md Framework	18
7.2 The Skill Architecture Framework	18
7.3 The Permission Framework	19
7.4 The Context Management Framework	19
Module 8: Playbook Workflows	19
8.1 The Daily Marketing Automation Workflow	19
8.2 The Client Onboarding Workflow	20
8.3 The Product Launch Workflow	20
8.4 Decision Tree: Choosing the Right Skill	20
Module 9: Common Mistakes	21
9.1 Skipping Plan Mode	21
9.2 Neglecting Context Management	21
9.3 Overloading the CLAUDE.md File	21
9.4 Using Bypass Permissions on Critical Systems	22
9.5 Writing Vague Skill Descriptions	22
Module 10: Advanced Strategies	22
10.1 Token Cost Optimization	22
10.2 Skill Chaining and Composition	22
10.3 Multi-Agent Client Management	23
10.4 Custom Evaluation Systems	23
10.5 Scaling from Individual to Team	23
Module 11: Templates and Checklists	24
11.1 Claude Code Setup Checklist	24
11.2 CLAUDE.md Template	24
11.3 Custom Skill Template	25
11.4 Essential Slash Commands Reference	25

11.5 Workflow Decision Checklist	26
Module 12: Key Takeaways	26
12.1 Actionable Insights	26
12.2 Key Mental Models	27

Module 1: Introduction

1.1 Why This Playbook Exists

The convergence of artificial intelligence and marketing has reached an inflection point. Claude Code, Anthropic's command-line AI agent, has emerged as one of the most powerful tools available for marketers, founders, and growth professionals who want to move beyond simple chatbot interactions and build real, production-grade systems. This playbook exists because the gap between knowing about Claude Code and actually using it effectively is wider than most people realize. The documentation is scattered across terminal manuals, blog posts, GitHub repositories, and developer forums. There is no single resource that bridges the non-developer's perspective with advanced implementation patterns, and that is exactly the void this playbook fills.

Claude Code is not just a coding assistant. It is a business infrastructure layer. It can scrape leads, generate proposals, build websites, create videos, and orchestrate complex multi-step marketing workflows directly from your terminal. When combined with structured agent skills, such as those found in Corey Haines' `marketingskills` repository, Claude Code transforms from a simple chat tool into a complete marketing operations platform. The skills cover conversion rate optimization, copywriting, SEO, paid advertising, analytics, churn prevention, and dozens of other disciplines, all executable through natural language commands. This playbook synthesizes the best insights from both primary sources into one coherent, actionable system that anyone can follow regardless of their technical background.

1.2 Who This Playbook Is For

This playbook is designed for three distinct audiences, each at a different stage of their AI marketing journey. The first group is non-technical marketers and founders who want to leverage AI for real business outcomes but feel intimidated by terminal interfaces and coding environments. If you have never opened a terminal before, the Beginner module will walk you through every step from installation to your first automated workflow. The second group is intermediate users who have experimented with Claude Code but want to unlock its more powerful capabilities, such as parallel agent orchestration, MCP server integrations, and skill-based automation. The third group is advanced practitioners and agency operators who want to scale Claude Code across multiple clients and complex projects, building custom skill libraries and integrating with enterprise tools through Composio and MCP protocols.

Regardless of your starting point, the playbook assumes you are motivated by results, not theory. Every section is structured around practical implementation, not conceptual explanation. The goal is that by the time you finish reading, you will have a clear mental model of how Claude Code works, a library of reusable skills and workflows, and the confidence to build custom automation systems that produce measurable business outcomes. Whether you are a solo founder looking to automate lead generation or an agency team deploying Claude Code across client accounts, this playbook provides the framework.

1.3 What Readers Will Achieve

After completing this playbook, you will possess a comprehensive understanding of Claude Code's architecture, permission system, and context management model. You will know how to set up and configure Claude Code with optimal CLAUDE.md files that prime every session with your project's context. You will be able to install and deploy agent skills from the marketingskills repository across six categories of marketing operations, from conversion rate optimization to paid advertising strategy. You will understand how to build custom skills that automate repetitive marketing tasks, and how to connect Claude Code to external tools like Google Sheets, Gmail, Figma, GA4, and HubSpot through connectors and MCP servers. You will have practical workflows for lead scraping, proposal generation, website creation, video production, SEO auditing, and competitive analysis, all executable from a single terminal command.

Beyond individual workflows, you will also develop the strategic thinking required to scale Claude Code within an organization. The advanced modules cover parallel agent orchestration for handling multiple client projects simultaneously, error recovery patterns for when builds go wrong, and optimization techniques for reducing token costs while maximizing output quality. You will leave with a toolkit of checklists, templates, and decision trees that you can immediately deploy in your own work.

Module 2: Concept Foundations

2.1 What Is Claude Code?

Claude Code is a command-line tool developed by Anthropic that runs locally on your machine. Unlike browser-based AI chatbots, Claude Code operates within your file system, reading, writing, creating, and modifying files directly. It can call APIs, browse the web, spawn sub-agents to run research in parallel, and execute scripts in Python, JavaScript, and other languages. Think of it as an AI agent that lives inside your computer and works with your existing tools rather than around them. It is not a cloud service that processes requests in isolation; it is a local agent with direct access to your project files, terminal commands, and development environment.

The distinction between Claude Code and Anthropic's browser-based Claude is important. While both use the same underlying models, Claude Code has capabilities that the web interface does not: it can execute terminal commands, manage git repositories, install packages, run test suites, and interact with your local file system. This makes it fundamentally different from a chat interface. Claude Code is not just answering questions; it is performing actions. It can create a full Next.js application, set up a database schema, deploy a website, or orchestrate a complex multi-step marketing workflow, all from natural language instructions typed into your terminal.

2.2 What Are Agent Skills?

Agent skills are the bridge between Claude Code's general capabilities and specialized marketing tasks. A skill is essentially a Markdown file with YAML frontmatter that sits in a specific folder within your project. When Claude Code reads a skill file, it gains domain-specific knowledge and a step-by-step workflow for a particular task. Instead of explaining to Claude Code from scratch how to audit a website's SEO, you install a pre-built skill that contains all the methodology, checklists, and best practices. One invocation runs the entire workflow. The skill architecture follows the Agent Skills specification at agentskills.io, which defines a cross-agent standard compatible with Claude Code, OpenAI Codex, Cursor, Windsurf, and any other AI coding agent that supports the spec.

Corey Haines' `marketingskills` repository contains 33 such skills organized across ten marketing categories. Each skill is a self-contained Markdown file under 500 lines with a structured format that includes the skill name, description, trigger phrases, and detailed step-by-step instructions. The foundation skill, called `product-marketing-context`, is read by all other skills first to establish the baseline context about your product, audience, and positioning. This layered architecture means that when you invoke any marketing skill, Claude Code already understands your business context before it begins executing the specific workflow.

2.3 The MCP Protocol

MCP (Model Context Protocol) is Anthropic's open standard for connecting AI agents to external tools and data sources. An MCP server is a lightweight service that Claude Code can connect to, giving it capabilities beyond its built-in toolset. For example, connecting the Playwright MCP server gives Claude Code browser automation capabilities. It can open websites, click buttons, fill forms, take screenshots, and verify that web applications work correctly. Connecting the Context7 MCP server allows Claude Code to fetch up-to-date library documentation, preventing it from hallucinating outdated API methods. The Excalidraw MCP server lets Claude Code generate architecture diagrams and flowcharts as hand-drawn sketches. DeepWiki provides structured documentation for any GitHub repository.

The practical significance of MCP is that it transforms Claude Code from a text-based assistant into a tool-agnostic operator. With the right MCP servers active, Claude Code can interact with databases, browsers, design tools, analytics platforms, and communication systems. The `marketingskills` repository includes 51 zero-dependency Node.js CLI tools for marketing platforms like GA4, Google Ads, Meta Ads, HubSpot, Klaviyo, and Mailchimp, plus 40+ API integration guides with MCP server support and Composio integration for OAuth-heavy tools. Composio acts as a bridge layer, providing MCP access to tools that require OAuth authentication without native MCP servers, covering platforms like HubSpot, Salesforce, Meta Ads, LinkedIn Ads, Google Sheets, Slack, and Notion.

2.4 Context Windows and Token Economics

Context is the single most important scarce resource in Claude Code. Every session has a finite context window, measured in tokens, that determines how much information Claude Code can process at once. System prompts, tool definitions, MCP server configurations, skill files, your CLAUDE.md, and your conversation history all consume tokens from this window. Before you type a single message, you are often already 15-20% through your context window from system-level overhead alone. Managing context effectively is the difference between a smooth, productive build and a degraded session where Claude Code loses track of instructions and produces inconsistent output.

The key principle is proactive context management. Use the `/compact` command at around 50% context usage, not 95%. By the time you reach 95%, the quality of Claude Code's output has already degraded significantly because earlier instructions have been compressed or truncated. The `/context` command shows you exactly what is consuming your window, with exact token counts for each component. The `/clear` command resets context entirely when switching to an unrelated task. Plan Mode is another context optimization strategy: spending five minutes planning before building prevents the costly cycle of building incorrectly, rebuilding, and consuming double the tokens. Every hour saved in complex builds comes from spending five minutes in Plan Mode first.

Module 3: System Overview

3.1 The Claude Code Architecture

Claude Code's architecture can be understood as four interconnected layers, each building on the one below it. At the foundation is the Local Execution Layer, which provides Claude Code with direct access to your file system, terminal commands, and local development environment. This is what enables it to create files, install packages, run scripts, and execute terminal commands. Above that sits the Context Layer, which manages the CLAUDE.md file, skill files, conversation history, and system prompts. This layer determines what information Claude Code has available at any given moment and how efficiently it uses its token budget. The third layer is the Tool Integration Layer, which encompasses MCP servers, connectors, CLI tools, and API integrations. This is the bridge between Claude Code and external systems like analytics platforms, email services, and design tools. At the top is the Orchestration Layer, which handles parallel agent execution, task decomposition, and multi-step workflow management.

Layer	Function	Key Components
Local Execution	File system access, terminal commands, script execution	Terminal, file system, package managers
Context Management	Project knowledge, conversation state, token budget	CLAUDE.md, skills, conversation history, <code>/compact</code>

Tool Integration	External platform connectivity, data access	MCP servers, connectors, CLI tools, Composio
Orchestration	Multi-step workflows, parallel agents, task routing	Sub-agents, Plan Mode, skill chaining

These layers work together in every interaction. When you ask Claude Code to perform a marketing task, the Orchestration Layer decomposes the request into steps, the Tool Integration Layer connects to the necessary external platforms, the Context Layer ensures the right skills and project knowledge are loaded, and the Local Execution Layer carries out the actual file operations and script executions. Understanding this architecture helps you diagnose problems when something goes wrong. If Claude Code is producing irrelevant output, the issue is likely in the Context Layer. If it cannot access a tool, the issue is in the Tool Integration Layer. If it is not executing commands properly, the issue is in the Local Execution Layer.

3.2 The Agent Skills Ecosystem

The marketingskills repository represents a complete marketing operations toolkit for AI agents. The 33 skills span ten categories and follow a hierarchical dependency structure. At the foundation is the product-marketing-context skill, which every other skill reads first. This ensures that when any marketing task is executed, Claude Code already understands your product, audience, value proposition, competitive positioning, and key messaging. Without this foundation, each skill would need to be briefed on your business context every time, wasting tokens and producing generic output.

Category	Skills	Purpose
Conversion Optimization	page-cro, signup-flow-cro, onboarding-cro, form-cro, popup-cro, paywall-upgrade-cro	Optimize every conversion touchpoint
Content and Copy	copywriting, copy-editing, cold-email, email-sequence, social-content, lead-magnets	Create and refine marketing content
SEO and Discovery	seo-audit, ai-seo, programmatic-seo, site-architecture, competitor-alternatives, schema-markup, aso-audit	Maximize organic visibility
Strategy	marketing-ideas, marketing-psychology, launch-strategy, pricing-strategy	Plan and position for growth
Paid and Measurement	paid-ads, ad-creative, analytics-tracking, ab-test-setup, community-marketing	Allocate spend and measure results
Sales and RevOps	revops, sales-enablement	Align marketing and revenue

Growth	free-tool-strategy, referral-program	Build sustainable acquisition channels
Retention	churn-prevention	Reduce attrition and recover revenue
Foundation	product-marketing-context	Provide baseline business context

3.3 How the Pieces Connect

The connection between Claude Code, agent skills, MCP servers, and external tools follows a clear data flow pattern. You begin by defining your project context in the CLAUDE.md file and installing the relevant marketing skills. When you invoke a skill, Claude Code reads the skill instructions, loads any cross-referenced skills, and begins executing the workflow. If the skill requires data from external platforms, it routes through MCP servers or CLI tools. If it needs to browse the web, it uses built-in web search. If it needs to process files, it operates directly on your file system. The output flows back through the same layers: files are written locally, data is pushed to connected platforms, and reports are generated as structured documents.

The practical implication is that you can build end-to-end marketing pipelines. A lead generation pipeline might start with the competitor-alternatives skill to identify target keywords, flow into the programmatic-seo skill to generate landing pages, connect to Google Ads through the paid-ads skill and MCP integration, and feed results into the analytics-tracking skill for measurement. Each skill feeds the next, with Claude Code orchestrating the transitions. The connector layer amplifies this further. Connect Google Sheets and lead scraping uploads results directly. Connect Gmail and email classification reads and labels your real inbox. Connect Figma and your built interfaces land on the design canvas as editable frames. The skill provides the workflow; the connector provides the live data and destination.

Module 4: Step-by-Step Implementation

4.1 Beginner Workflow

Step 1: Acquire a Claude Plan

You need at least Claude Pro at \$20 per month. Claude Max gives more usage for heavy workflows. The Pro tier provides sufficient access for learning the interface, building small projects, and running individual skills. If you plan to use Claude Code extensively for production workflows with parallel agents and long build sessions, the Max tier is recommended. There is also a free option using local open-source models through Ollama: install Ollama, pull a model like qwen3-coder or gpt-oss:20b, and point Claude Code at your local endpoint. This requires at least 32GB of RAM and produces noticeably lower quality output than Claude's own models. It is suitable for learning the interface but not for the

multi-step workflows described later in this playbook.

Step 2: Install Claude Code

Open your terminal. On Mac, press Command+Space to open Spotlight, type "terminal," and press Enter. On Windows, open CMD or PowerShell. Run the installation command from code.claude.com/docs. The process takes about two minutes. If the terminal is unfamiliar, you can use Claude Code through an IDE instead. The two recommended options are VS Code, which is free from Microsoft and available at code.visualstudio.com, and Windsurf, which is newer and more AI-native. Both let you run Claude Code through a graphical chat panel on the right side of your screen while your files sit on the left. To add Claude Code to VS Code, go to the Extensions tab, search for "Claude Code," install the extension from Anthropic, and click the Claude icon that appears in your editor panel.

Step 3: Authenticate and Run

Type "claude" in your terminal. It will prompt you to log in with your Claude account. Follow the authentication steps. Then navigate to any project folder using the "cd" command and type "claude" again. You are now running Claude Code in that project directory. Claude Code can read every file in the folder, create new files, modify existing ones, and execute terminal commands, all based on your natural language instructions. Type a message like "What files are in this directory?" to verify everything is working, then proceed to create your first CLAUDE.md file.

Step 4: Create Your CLAUDE.md File

The CLAUDE.md file is the single highest-leverage configuration in Claude Code. It sits in your project folder and is read automatically at the start of every session. Think of it as the standing brief for your project: it tells Claude Code what the project is for, how you want it to behave, what it should never do, the tools and conventions you use, and any constraints specific to this work. Every session with Claude Code starts cold with no memory of previous interactions. The CLAUDE.md is what makes it feel like a tool that knows your work rather than a blank slate you brief every morning.

To generate your first CLAUDE.md automatically, run `/init` in any project folder. Claude Code will read through your existing files and produce a summary with working instructions. Then refine it over time using these rules: keep the most important constraints at the very top because Claude Code has primacy bias and the first things it reads stick hardest. Write in bullet points and short headings for high information density. Keep the file between 200 and 500 lines maximum. Never paste entire API documentation into it; give Claude Code a summary and let it fetch the rest when required. Update it whenever Claude Code keeps making the same mistake. If it trips on the same issue twice, add a rule.

Step 5: Understand Permission Modes

Claude Code has four permission modes, and understanding them fundamentally changes how you use the tool. "Ask Before Edits" is the default mode, where Claude Code asks permission before changing

any file. It is safe but slow, especially during complex builds where it asks you to approve every individual edit. "Edit Automatically" auto-accepts changes to existing files but still asks before creating new ones. This is a good middle ground for most work. "Bypass Permissions" removes all prompts by adding the flag `--dangerously-skip-permissions` when launching Claude Code from your terminal, typing `"claude --dangerously-skip-permissions"` instead of just `"claude."` Most experienced users end up here for knowledge work. The risk is real but low if you work in a non-critical folder.

The fourth mode is "Plan Mode," which is the one most beginners undervalue. Plan Mode is read-only. Claude Code researches your project, reads your files, checks documentation, and produces a plan before executing anything. The plan is load-bearing, not cosmetic. The math is straightforward: build without planning, it is wrong, rebuild, total cost is build time plus rebuild time plus tokens consumed by both attempts. Plan first, Claude Code identifies the approach problem before touching a single file, total cost is five minutes of planning plus one correct build. To use it, click "Plan Mode" in the permission toggle at the bottom of the chat panel, describe what you want to build, let Claude Code produce the plan, read it, push back if something looks off, then switch to Bypass Permissions when the plan looks right and let it build.

4.2 Intermediate Workflow

Step 6: Install Marketing Skills

With the basics established, the next step is to install the marketing skills from Corey Haines' `marketingskills` repository. The recommended method is using `npx skills`, the Vercel Labs skills installer. Open your terminal and run: `npx skills add coreyhaines31/marketingskills`. This command downloads all 33 skills and installs them into your `.agents/skills/` directory with automatic symlinks to `.claude/skills/` for Claude Code compatibility. Alternative installation methods include the Claude Code Plugin system using the command `/plugin marketplace add coreyhaines31/marketingskills`, manual git clone and copy, git submodules for easy updates, forking and customizing for your specific needs, and SkillKit for multi-agent setups using `npx skillkit install coreyhaines31/marketingskills`.

Step 7: Configure the Product Marketing Context

Before using any marketing skill, you must configure the `product-marketing-context` foundation skill. This skill is read by every other marketing skill before execution, providing Claude Code with essential context about your business. Navigate to the `.agents/skills/product-marketing-context/` directory and review the `SKILL.md` file. It contains structured fields for your product description, target audience, value proposition, competitive positioning, key differentiators, pricing model, and brand voice. Fill in each field with specific, detailed information. Generic descriptions produce generic output. The more context you provide, the more tailored and effective every subsequent skill invocation will be. This is a one-time setup that pays dividends across every marketing task.

Step 8: Install MCP Servers

Four MCP servers are recommended as first installations. Context7 fetches up-to-date library documentation into context, stopping Claude from hallucinating outdated APIs. Playwright gives Claude browser automation, enabling it to implement, test, and verify UI features by navigating your app itself. Excalidraw lets Claude generate architecture diagrams and flowcharts as hand-drawn sketches from a prompt. DeepWiki pulls structured documentation for any GitHub repo, allowing Claude to understand external codebases without manual README pasting. To install these, use the `claude mcp add` command in your terminal. Each MCP server connects Claude Code to a new capability domain, expanding the range of marketing tasks it can perform autonomously.

Step 9: Connect External Tools

Go to Settings, then Connectors in Claude Desktop. Over 50 integrations are listed. Authenticate once, and every future session picks them up automatically. The most impactful connections for marketing workflows include Google Sheets for data output and lead management, Gmail for email classification and outreach, Figma for design collaboration, and GA4 for analytics data. With connectors active, Claude Code can write lead scraping results directly into Google Sheets, classify and label incoming emails in Gmail, and push built interfaces to Figma as editable design layers. The connector transforms skills from local file operations into integrated business workflows that connect to the tools your team already uses daily.

4.3 Advanced Workflow

Step 10: Build Custom Skills

Creating your own skills is where Claude Code transitions from a tool you use into a system you build upon. Each custom skill is a Markdown file with YAML frontmatter stored in your `.claude/skills/` directory. The `SKILL.md` file must be under 500 lines and follow a specific format. The YAML frontmatter contains two required fields: "name," which must match the directory name, and "description," which should include trigger phrases and keywords that tell Claude Code when to use the skill. The body of the file contains step-by-step instructions written in direct, instructional language with H2 and H3 headings for structure. A well-written skill reads like a standard operating procedure that Claude Code executes mechanically.

The skill structure template includes four optional subdirectories: `references/` for additional documentation, `scripts/` for executable code, and `assets/` for templates, images, and data files. When building a custom skill, start by identifying the exact sequence of steps you want Claude Code to follow. Write each step as a clear, imperative instruction. Include decision criteria where Claude Code needs to make judgments, such as when to escalate a task or when to skip a step based on conditions. Test the skill by invoking it in a fresh Claude Code session and observing whether it follows the intended workflow. Iterate on the instructions based on the results. The quality of your skills directly determines

the quality of Claude Code's output.

Step 11: Orchestrate Parallel Agents

For complex marketing operations that involve multiple simultaneous tasks, Claude Code can spawn sub-agents that run research and execution in parallel. This is the intermediate-to-advanced capability that transforms Claude Code from a single-threaded assistant into a multi-threaded operations platform. For example, a comprehensive competitive analysis might involve one agent scraping competitor websites, another analyzing their SEO profiles, a third reviewing their ad creative, and a fourth synthesizing the findings into a report. Each agent runs independently, accessing its own context and tools, and the results are merged into a coherent output.

To use parallel agents, structure your Claude Code instruction to explicitly request sub-agent delegation. Claude Code will break down the task, spawn the necessary agents, monitor their progress, and aggregate the results. The key optimization is context allocation: each sub-agent gets its own context window, so you avoid the token bottleneck of a single session. For agencies managing multiple clients, parallel agents enable you to run client-specific workflows simultaneously without context contamination between accounts. This is where Claude Code stops being a productivity tool and becomes a genuine business infrastructure layer that scales with your operations.

Step 12: Integrate Enterprise Tools via Composio

Composio provides MCP access to OAuth-heavy tools that do not have native MCP servers. This covers platforms like HubSpot, Salesforce, Meta Ads, LinkedIn Ads, Google Sheets, Slack, and Notion. The integration works through a middleware layer that handles OAuth authentication and exposes each platform's API through a standardized MCP interface. To set up Composio, install the MCP server, authenticate with each platform through Composio's dashboard, and configure the connection in Claude Code. Once connected, Claude Code can perform operations on these platforms as if they had native MCP servers. This is particularly valuable for enterprise marketing stacks where HubSpot, Salesforce, or similar platforms are central to marketing and sales operations.

Module 5: Real-World Use Cases

5.1 Lead Scraping and Enrichment

Lead scraping is one of the most immediately valuable use cases for Claude Code in a marketing context. The workflow uses a custom skill that scrapes a target industry, classifies leads with AI, enriches contact emails, and uploads everything to a Google Sheet. The skill accepts three inputs: industry, location, and the number of leads required. Claude Code uses web search to find businesses matching the criteria, classifies each lead with structured data including business name, website, phone number, and category, then enriches contact email addresses where they are publicly available. The

entire dataset is uploaded to a connected Google Sheet for immediate use by the sales team.

To build this skill, you provide Claude Code with a prompt that describes the complete workflow as a step-by-step checklist. The skill file is created in the `.claude/skills/` directory and can be invoked with a single line: "Use the lead-scraping skill. Industry: dentists. Location: Manchester, UK. Leads required: 20. Upload to this Google Sheet: [YOUR SHEET URL]." Claude Code executes every step autonomously, from the initial web search through to the final Google Sheets upload. The quality of the output depends on the specificity of your input parameters and the quality of the web data available for the target industry and location.

5.2 Proposal Generation

The proposal generation workflow demonstrates Claude Code's ability to create complete web applications from natural language descriptions. The workflow produces a full proposal generator with a brief input form, AI-generated proposals written in your brand voice, a client-facing view with an Accept button, and a payment link field. There is no PandaDoc subscription and no per-proposal cost. The entire system is built as a local web application using plain HTML, CSS, and JavaScript with no frameworks required.

The recommended approach is to use Plan Mode first. Switch to Plan Mode, then paste your requirements: build a proposal generator with an input form for client name, industry, scope, and budget, AI-generated proposal output written in your voice using a `brand-voice.md` reference file, a client-facing view with an Accept button, and a payment link field. Claude Code produces the plan, which you review and refine. Once the plan is solid, switch to Bypass Permissions and let Claude Code build the entire application without interruption. Each proposal is saved as a Markdown file in a `/proposals` folder. The system integrates with your existing brand guidelines and can be customized to match any company's visual identity and messaging framework.

5.3 Website Generation from Screenshots

Claude Code can generate complete, deployable websites from a screenshot of a site you admire combined with your own content and brand guidelines. The workflow involves dragging a reference screenshot into your terminal, providing your content including headline, subheadline, key points, CTA text, and brand colour, and instructing Claude Code to build a complete single-page website using your content but matching the visual style of the reference. Claude Code produces three layout variants as separate HTML files with embedded CSS, which you can review and iterate on.

To push the finished build into Figma for design refinement, connect the Figma MCP server first using the command: `claude mcp add --transport http figma https://mcp.figma.com/mcp`. Then instruct Claude Code to start a local server for your app and capture the UI in a new Figma file. Claude Code opens a browser, captures the live interface, and sends it to Figma as editable design layers. You can refine the design on the Figma canvas and push updates back. Design and code stay in sync without manual

handoff. This workflow eliminates the traditional disconnect between design mockups and implemented code, creating a seamless design-to-development pipeline.

5.4 Video Generation with Remotion

Remotion is an open-source framework that lets you create videos programmatically using React. Every frame is a component, and every animation is code. Claude Code writes the React components that render your video, giving you full control, full editability, and the ability to generate variations from a single template. This is fundamentally different from AI generating random video from a text prompt. The setup process involves four steps: create a new Remotion project using `npx create-video@latest`, navigate into the folder and install dependencies, start the preview server with `npm run dev`, open a second terminal window and start Claude Code, then describe the video you want. Claude Code reads your project, writes the components, and the preview at `localhost:3000` updates in real time.

The quality of your output depends almost entirely on the quality of your source material. Good screen recordings produce good videos. Blurry screenshots produce generic results. When you are satisfied with the result, render your final MP4 using `npx remotion render`. Note that Remotion is free for personal use and open source projects, but companies with three or more employees need a commercial license to render videos for clients. Check remotion.dev/license before building client work.

5.5 SEO Audit and Optimization

The seo-audit skill from the marketingskills repository provides a comprehensive technical and on-page SEO audit framework. When invoked, Claude Code systematically analyzes a target website across multiple dimensions including crawlability and indexation, site architecture and URL structure, on-page optimization for target keywords, technical performance metrics, schema markup implementation, internal linking structure, and mobile responsiveness. The audit produces a structured report with prioritized recommendations organized by impact and implementation effort. Combined with the schema-markup skill for implementing structured data and the ai-seo skill for optimizing for AI search experiences including AI Overviews, this creates a complete SEO optimization pipeline.

Module 6: Tools and Stack

6.1 Core Platform

Claude Code itself is the central platform, but the ecosystem extends across several categories of supporting tools. The core platform includes Claude Code (the command-line agent), Claude Desktop (for connector management), and Claude Pro or Max subscriptions (for API access). The development environment can be a raw terminal, VS Code with the Claude Code extension, or Windsurf for a more

AI-native experience. Each option has trade-offs: the terminal provides the most direct control, VS Code adds a visual file browser and integrated chat panel, and Windsurf offers a more opinionated AI-first workflow that some users prefer.

6.2 MCP Servers

MCP Server	Capability	Why It Matters
Context7	Fetches up-to-date library documentation	Prevents Claude from hallucinating outdated APIs
Playwright	Browser automation for testing	Claude can navigate, click, fill forms, and verify UI
Excalidraw	Architecture diagrams and flowcharts	Visual documentation from natural language prompts
DeepWiki	Structured docs for any GitHub repo	Understand external codebases without manual research
Figma	Design file integration	Push and pull designs between code and canvas

6.3 Marketing Tool Integrations

The marketingskills repository includes 51 zero-dependency Node.js CLI tools covering the major marketing platforms. These CLI tools enable Claude Code to interact with platforms like Google Analytics 4, Google Ads, Meta Ads, LinkedIn Ads, HubSpot, Klaviyo, Mailchimp, Stripe, Amplitude, Mixpanel, Hotjar, Ahrefs, and dozens more. Each tool supports a `--dry-run` flag for previewing requests before execution, and the tools have been security-hardened with header-based authentication, URL encoding, and input validation. They have been reviewed through an independent Codex security audit. The integration guides in the repository provide step-by-step setup instructions for each platform, including API key configuration, authentication setup, and example queries.

6.4 Composio for OAuth-Heavy Platforms

Composio provides the MCP integration layer for platforms that require OAuth authentication, which includes most enterprise marketing and sales tools. Without Composio, connecting Claude Code to HubSpot, Salesforce, Meta Ads, or LinkedIn Ads would require building custom OAuth flows. Composio handles this complexity through a standardized middleware layer. After installing the Composio MCP server and authenticating with each platform through Composio's dashboard, Claude Code can perform authenticated operations on these platforms as if they had native MCP servers. This dramatically expands the range of enterprise marketing operations that can be automated through Claude

Code.

Module 7: Framework Breakdowns

7.1 The CLAUDE.md Framework

The CLAUDE.md file is the foundational framework for Claude Code mastery. It operates on the principle of primacy bias: Claude Code gives the most weight to the first information it encounters in a session. This means the most critical constraints must appear at the top of the file. The framework has five components: project definition, behavioral rules, tool conventions, constraints and restrictions, and error correction rules. The project definition answers what the project is, who it is for, and what success looks like. Behavioral rules define how Claude Code should communicate and make decisions. Tool conventions specify which frameworks, libraries, and technologies to use. Constraints list what Claude Code must never do. Error correction rules address recurring mistakes.

The optimal CLAUDE.md is between 200 and 500 lines. Shorter than 200 lines and you are not providing enough context for complex tasks. Longer than 500 lines and you are paying token costs that reduce output quality because the context window fills up with instructions before any actual work begins. The format should use bullet points and short headings for maximum information density. Never paste entire API documentation into the file. Instead, give Claude Code a summary and let it fetch the specific documentation it needs when required. Update the file iteratively: whenever Claude Code makes the same mistake twice, add a specific rule to prevent it from happening again.

7.2 The Skill Architecture Framework

The Agent Skills specification defines a standard format for creating reusable AI agent capabilities. Each skill has a defined structure: a SKILL.md file with YAML frontmatter (name and description fields are required), optional subdirectories for references, scripts, and assets, and a maximum length of 500 lines. The description field is particularly important because it includes trigger phrases that tell Claude Code when to activate the skill. For example, a lead-scraping skill description might include phrases like "find leads," "scrape businesses," "prospect list," or "build a database of prospects." When Claude Code encounters these phrases in your instructions, it automatically loads the skill and follows its workflow.

Skills follow a dependency hierarchy. The product-marketing-context skill is the foundation that all other marketing skills read first. Beyond that, skills cross-reference each other based on workflow dependencies: the copywriting skill references the page-cro skill for conversion-optimized copy, the revops skill references sales-enablement for lead routing, and the seo-audit skill references schema-markup for technical implementation. When you invoke a skill, Claude Code automatically loads any cross-referenced skills, creating a chain of domain expertise that produces comprehensive output.

7.3 The Permission Framework

Claude Code's permission system follows a trust escalation model with four levels. At the first level, "Ask Before Edits," every file modification requires explicit approval. This is appropriate for learning and for working with critical files where mistakes are costly. At the second level, "Edit Automatically," modifications to existing files are approved but new file creation still requires permission. This is the recommended mode for day-to-day work. At the third level, "Bypass Permissions," all operations proceed without interruption. This is the mode experienced users adopt for knowledge work in non-critical folders. The fourth level, "Plan Mode," is not a permission level per se but a read-only planning state that prevents any file modifications while Claude Code researches and plans.

The decision framework for choosing a permission level depends on three factors: the criticality of the files you are working with, your confidence in the task, and the complexity of the workflow. For production systems or files that cannot be easily reverted, stay at level one or two. For experimental work, prototyping, and content generation in disposable project folders, use level three. Always use Plan Mode before any complex task regardless of your default permission level. The combination of Plan Mode followed by Bypass Permissions is the workflow that most experienced Claude Code users adopt for complex builds: plan carefully, then execute without interruption.

7.4 The Context Management Framework

Effective context management follows a lifecycle pattern: load, execute, compact, and clear. During the load phase, Claude Code reads your CLAUDE.md, any activated skills, and MCP server configurations. During the execute phase, your conversation history grows and consumes tokens. The compact phase should be triggered proactively at 50% context usage using the `/compact` command, not reactively at 95%. The clear phase resets context entirely using `/clear` when switching between unrelated tasks. The critical slash commands for context management are: `/context` to inspect current usage, `/compact` to free up space, `/clear` to reset, `/model` to switch models mid-session, and `/rewind` (or pressing Escape twice) to undo when Claude Code goes off track.

Module 8: Playbook Workflows

8.1 The Daily Marketing Automation Workflow

This workflow is designed for marketing teams who want to integrate Claude Code into their daily operations. It consists of five phases that run each morning. Phase one is content scanning: use the web search capability to scan industry news sources and competitor activity, summarizing key developments. Phase two is social content generation: invoke the social-content skill to generate a day's worth of social media posts based on current trends and your brand voice. Phase three is email triage: use the email

classification capability to scan your inbox, label priority messages, and draft responses for routine inquiries. Phase four is analytics review: connect to GA4 through the analytics-tracking skill and request a summary of yesterday's performance metrics. Phase five is task prioritization: based on the analytics review and competitive intelligence, generate a prioritized task list for the day.

8.2 The Client Onboarding Workflow

For agencies and consultants, Claude Code can automate significant portions of the client onboarding process. Step one: use the customer-research skill to analyze the client's industry, competitors, and market position. Step two: run the seo-audit skill on the client's existing website to establish a performance baseline. Step three: use the page-cro skill to identify quick-win conversion optimization opportunities on key landing pages. Step four: invoke the marketing-ideas skill filtered to the client's industry to generate a prioritized list of growth strategies. Step five: compile all findings into a structured onboarding report saved as a Markdown file. This workflow takes approximately fifteen minutes of Claude Code execution time and produces a deliverable that would traditionally require several hours of analyst work.

8.3 The Product Launch Workflow

Launching a new product or feature involves coordinated execution across multiple marketing channels. The launch-strategy skill provides the framework, and Claude Code orchestrates the execution. Begin with the launch-strategy skill to develop the launch plan, including timeline, channels, messaging pillars, and success metrics. Then use the copywriting skill to create landing page copy, email announcements, and social media content. Use the paid-ads skill to set up launch campaigns across Google Ads and Meta. Use the email-sequence skill to create a pre-launch teaser sequence, a launch day announcement, and a post-launch follow-up series. Use the analytics-tracking skill to ensure all conversion events are properly instrumented before launch day. Each of these steps produces structured output that feeds into the next, creating a complete launch playbook.

8.4 Decision Tree: Choosing the Right Skill

With 33 skills available, choosing the right one can be challenging. Use this decision framework: if the task involves improving conversion rates on existing pages, use the appropriate CRO skill (page-cro, signup-flow-cro, onboarding-cro, form-cro, popup-cro, or paywall-upgrade-cro based on the specific funnel stage). If the task involves creating new content, use copywriting for pages, cold-email for outreach, email-sequence for automated flows, social-content for social platforms, or lead-magnets for gated content. If the task involves organic search visibility, use seo-audit for auditing, ai-seo for AI search optimization, programmatic-seo for scaled page generation, or schema-markup for structured data. If the task involves paid channels, use paid-ads for campaign setup, ad-creative for creative generation, or ab-test-setup for experimentation. If the task involves strategy, use marketing-ideas for

ideation, marketing-psychology for behavioral insights, launch-strategy for launches, or pricing-strategy for monetization.

Module 9: Common Mistakes

9.1 Skipping Plan Mode

The most common mistake beginners make is jumping straight into building without spending time in Plan Mode. The temptation is understandable: you have a clear idea of what you want, and you want Claude Code to start building immediately. But the token economics are unforgiving. Building without planning means you consume tokens on an incorrect approach, then consume more tokens on a rebuild. The total cost is always higher than planning first. Five minutes of planning consistently saves thirty minutes of building and rebuilding. Plan Mode is not a preliminary step; it is a load-bearing part of the workflow. Treat it as mandatory for any task that involves more than three files or any task where you are unsure about the approach.

9.2 Neglecting Context Management

Running `/compact` too late is the second most common mistake. Most beginners wait until Claude Code's output quality has noticeably degraded before compacting, which means they are already operating at reduced capacity. The rule is simple: compact at 50%, not 95%. By the time you reach 95%, earlier instructions have been compressed or truncated, and Claude Code has lost track of important context. Running `/context` regularly to monitor your usage is a habit that separates effective Claude Code users from frustrated ones. Another related mistake is not using `/clear` when switching between unrelated tasks. If you spend the morning working on a website build and then switch to a lead scraping task without clearing context, the residual context from the website build consumes tokens and may confuse Claude Code's output on the new task.

9.3 Overloading the CLAUDE.md File

Some users treat the CLAUDE.md file as a dumping ground for every piece of information they think might be relevant. The result is a thousand-line file that consumes 30% of the context window before any work begins. This leaves less room for conversation, skill content, and tool outputs, all of which degrade the quality of Claude Code's responses. Keep your CLAUDE.md between 200 and 500 lines. Give Claude Code a summary of your API documentation, not the entire documentation. Include only the constraints and rules that Claude Code actually needs. If a piece of information is only relevant for specific tasks, put it in a skill file instead of the CLAUDE.md, so it is only loaded when needed.

9.4 Using Bypass Permissions on Critical Systems

The bypass permissions mode is powerful, but using it on critical systems without proper safeguards is a recipe for data loss. There was a widely shared incident where someone running bypass permissions on a Linux machine lost data: the command Claude Code executed was valid, but the context was wrong. The lesson is to always work in non-critical folders when using bypass permissions, to use git for version control so you can revert changes, and to never use bypass permissions on production systems, databases, or irreplaceable files. The `/permissions` command provides a middle ground: you can pre-allow specific safe commands instead of bypassing all permissions globally.

9.5 Writing Vague Skill Descriptions

When creating custom skills, the description field in the YAML frontmatter is critical because it determines when Claude Code activates the skill. Vague descriptions like "helps with marketing" or "content tool" are too broad and either trigger at inappropriate times or fail to trigger when needed. Good descriptions include specific trigger phrases, the exact task the skill performs, and clear boundary conditions. For example, instead of "helps with SEO," write: "Use this skill when the user asks to audit a website's search engine optimization, analyze technical SEO issues, review on-page optimization, or identify keyword opportunities." The specificity ensures Claude Code activates the skill at the right moment and follows the correct workflow.

Module 10: Advanced Strategies

10.1 Token Cost Optimization

Advanced Claude Code users develop an intuitive sense for token economics. Every character in your `CLAUDE.md`, every skill loaded, every line of conversation history consumes tokens. The optimization strategy involves three levers. First, minimize base overhead: keep your `CLAUDE.md` lean, only load the skills you need for the current task, and use `/clear` when switching between unrelated projects. Second, maximize information density: use bullet points instead of prose in your `CLAUDE.md`, write skills with imperative instructions rather than explanatory text, and include only the data that Claude Code needs for the specific task at hand. Third, plan before executing: Plan Mode consumes fewer tokens than a failed build followed by a rebuild. The planning phase typically uses 10-15% of the tokens that a full build attempt would consume, making it the most cost-effective insurance against wasted tokens.

10.2 Skill Chaining and Composition

Individual skills are powerful, but chaining them together creates compound workflows that are far more valuable than the sum of their parts. Skill chaining works by using the output of one skill as the input for the next. For example, chain the competitor-alternatives skill to identify target keywords, feed those keywords into the programmatic-seo skill to generate landing pages, then use the analytics-tracking skill to instrument the new pages. Each skill builds on the previous one, creating an end-to-end pipeline from research to deployment to measurement. To implement skill chaining, structure your Claude Code instructions to reference multiple skills in sequence. Claude Code automatically loads the dependencies and manages the context transitions between skills.

10.3 Multi-Agent Client Management

For agencies and consultants managing multiple clients, Claude Code's parallel agent capability enables simultaneous execution of client-specific workflows. The key is context isolation: each client project should have its own project folder with its own CLAUDE.md and product-marketing-context. When spawning parallel agents, each agent operates within its own context window, preventing cross-contamination between client data. A typical multi-agent workflow might involve one agent running a monthly SEO audit for Client A, another generating social content for Client B, and a third analyzing paid advertising performance for Client C, all running simultaneously. The results are aggregated into individual client reports stored in each project's output folder.

10.4 Custom Evaluation Systems

The marketingskills repository includes 197 automated evaluations across all 33 skills. These evals test whether Claude Code produces correct, high-quality output when following each skill. For teams building custom skills, establishing an evaluation system is essential for maintaining quality at scale. The evaluation pattern involves creating test cases with known expected outputs, running Claude Code with the skill, and comparing the actual output against the expected output. The validate-skills.sh script in the repository provides a ready-made framework for skill validation. Running evals regularly catches regressions when skills are updated and ensures that custom modifications do not degrade output quality.

10.5 Scaling from Individual to Team

Transitioning from individual Claude Code usage to team-wide deployment requires addressing three challenges: consistency, knowledge sharing, and governance. Consistency is achieved by standardizing the CLAUDE.md format across all team projects and maintaining a shared skill library. Knowledge sharing is enabled by documenting successful workflows, custom skills, and CLAUDE.md patterns in a central repository that all team members can access. Governance involves establishing rules for what Claude Code can and cannot do, particularly around data access, client confidentiality, and external tool connections. The AGENTS.md file in the marketingskills repository provides a template for AI agent

guidelines that can be adapted for team use, covering expected behavior, coding standards, testing requirements, and content guidelines.

Module 11: Templates and Checklists

11.1 Claude Code Setup Checklist

#	Step	Details
1	Get Claude Pro or Max	Minimum \$20/month Pro plan required
2	Install Claude Code	Run install command from code.claude.com/docs
3	Authenticate	Type "claude" and follow login prompts
4	Install VS Code extension	Search "Claude Code" in Extensions tab
5	Run /init in project folder	Auto-generates initial CLAUDE.md
6	Customize CLAUDE.md	Add project context, rules, and constraints
7	Install MCP servers	Context7, Playwright, Excalidraw, DeepWiki
8	Connect external tools	Settings > Connectors in Claude Desktop
9	Install marketing skills	<code>npx skills add coreyhaines31/marketingskills</code>
10	Configure product context	Fill in product-marketing-context skill fields

11.2 CLAUDE.md Template

Below is a template structure for an effective CLAUDE.md file. Adapt it to your specific project needs, keeping it between 200 and 500 lines with the most important constraints at the top.

```
# Project: [Project Name]
# Type: [Web App / Marketing Site / Automation Tool]

## Core Rules (Primacy Position)
- [Most critical constraint #1]
- [Most critical constraint #2]
- [Most critical constraint #3]

## Project Context
- Purpose: [What this project does]
- Audience: [Who it is for]
- Tech Stack: [Frameworks and tools]
```

```

## Behavioral Rules
- Always [behavioral rule]
- Never [restriction]
- When in doubt, [decision heuristic]

## File Conventions
- Components in /src/components
- Styles in /src/styles
- Tests in /tests

## Error Corrections
- [Mistake Claude keeps making #1] -> [Correction rule]
- [Mistake Claude keeps making #2] -> [Correction rule]

```

11.3 Custom Skill Template

Use this template to create custom marketing skills for Claude Code. The directory name must match the name field in the frontmatter, and the SKILL.md must be under 500 lines.

```

---
name: your-skill-name
description: Use this skill when the user asks to [specific task].
Trigger phrases: [phrase 1], [phrase 2], [phrase 3]
---

# Skill: [Skill Title]

## Step 1: [First Step]
- [Instruction]
- [Instruction]

## Step 2: [Second Step]
- [Instruction]
- [If condition, then action]

## Step 3: [Third Step]
- [Instruction]
- [Instruction]

## Output Format
- [Expected output structure]

```

11.4 Essential Slash Commands Reference

Command	Function	When to Use
---------	----------	-------------

/plan	Start Plan Mode	Before any complex task with 3+ files
/context	Show context usage	When you want to check token consumption
/compact	Free up context	At 50% usage, NOT at 95%
/clear	Reset context	When switching to an unrelated task
/model	Switch models	When you need different reasoning level
/init	Generate CLAUDE.md	In a new project folder without one
/doctor	Diagnose issues	When Claude Code behaves unexpectedly
/rewind	Undo last action	When Claude Code goes off track (or Esc twice)
/permissions	Pre-allow commands	As alternative to full bypass mode
/btw	Side question	Ask without interrupting the main task

11.5 Workflow Decision Checklist

Before starting any Claude Code session, run through this decision checklist to ensure you are configured optimally for the task at hand. First, identify the task category: is this a content creation task, a data analysis task, a build task, or a research task? Second, load only the skills you need: do not activate all 33 marketing skills if you are only running an SEO audit. Third, verify your CLAUDE.md is relevant: if you are switching projects, run /clear and ensure the correct CLAUDE.md is loaded. Fourth, check context usage with /context. If you are above 30% before starting, consider /clear or /compact. Fifth, choose your permission level: use Plan Mode for complex tasks, Edit Automatically for routine modifications, and Bypass Permissions only in non-critical folders. Sixth, verify connector status: if your workflow requires Google Sheets or Gmail, confirm the connectors are active in Claude Desktop settings.

Module 12: Key Takeaways

12.1 Actionable Insights

- Claude Code is a business infrastructure layer, not just a coding tool. It reads, writes, and modifies files directly, calls APIs, browses the web, spawns sub-agents, and executes scripts. For marketers, this means you can automate entire operational workflows from a single terminal.
- The CLAUDE.md file is the single highest-leverage configuration you can make. Every session starts cold. The CLAUDE.md provides persistent context. Keep it between 200 and 500 lines, put critical constraints at the top, and update it whenever Claude Code repeats a mistake.

- Plan Mode saves more tokens than it costs. Five minutes of planning consistently prevents thirty minutes of rebuilding. Use it before any task involving three or more files.
- Compact context at 50%, not 95%. By the time you notice degraded output quality, you have already lost significant context. Run `/context` regularly and `/compact` proactively.
- Agent skills transform Claude Code from a general assistant into a specialist. The 33 marketing skills in the `marketingskills` repository cover the full spectrum of marketing operations. Install them with a single command and invoke them with a single line.
- MCP servers and connectors amplify Claude Code's capabilities exponentially. Without integrations, you are working on local files. With them, you are operating inside the tools your business already depends on: Google Sheets, Gmail, GA4, Figma, HubSpot, and dozens more.

12.2 Key Mental Models

Context as Currency: Every token in your context window is a resource. Spend it wisely on relevant information. Minimize overhead, maximize information density, and compact proactively. The user who manages context best gets the highest quality output from Claude Code.

Primacy Bias: Claude Code gives the most weight to the first information it encounters. This principle applies to your `CLAUDE.md` (put critical rules at the top), your conversation (lead with the most important instruction), and your skill descriptions (front-load trigger phrases).

Skills as Standard Operating Procedures: Think of each skill as a SOP that Claude Code executes mechanically. The quality of the SOP determines the quality of the output. Write skills with clear, imperative, step-by-step instructions. Include decision criteria and output formats.

The Connector Multiplier: A skill operating on local files is valuable. The same skill operating on live data through connectors is transformative. Prioritize connecting the tools your business uses daily. The return on investment is immediate and compounding.

Plan-Execute-Iterate: This three-phase cycle is the fundamental rhythm of productive Claude Code usage. Plan in Plan Mode, execute in Bypass Permissions, iterate based on the output. Skipping the planning phase is the most common mistake and the most expensive.